

# **TÉCNICAS DE COMPILACIÓN**

**Compilador para Subconjunto de C++**

Profesor:

- Ameri Francisco

Integrantes:

- Gomez Lucas
- Melillo Juan

# Introducción

El objetivo de este proyecto era crear un compilador capaz de realizar:

- Análisis Léxico
- Análisis sintáctico
- Análisis semántico
- Generación de Código Intermedio
- Optimización

Para completar este proyecto utilizamos un programa Java que implementa ANTL4 para que realice la compilación

Nuestro programa es capaz de realizar los 3 tipos de análisis, generara el código Intermedio, optimizarlo, y luego genera archivos de texto que demuestra dicho código intermedio y su optimización.

## Subconjunto de C++ Implementado

Nuestro compilador procesa un subconjunto de C++ de los datos que no parecían más específicos, Datos como:

- Datos Primitivos: Enteros, Caracteres, Booleanos o punto flotante (int, char, double, bool) y datos vacíos (void) para funciones sin retorno.
- Estructuras de Control:
  - Condicionales: if - else.
  - Bucles: while – for.
  - Control de Flujo: break – continue – return.
- Declaración de Variables: ya sea solo la variable o inicializándolas con un valor, e incluso funciones con parámetros.
- Operaciones Esenciales: como podrían ser:
  - Aritméticas: +, -, \*, /, %.
  - Lógicas: &&, ||, !.
  - Comparación: ==, !=, <, <=, >, >=.
  - Asignación: =.
- Variables Locales y Globales.
- Bloques de código delimitados por llaves {}.

# Diseño de la solución

La arquitectura general de compilador consta de 7 pasos:

- Código Fuente: Este es el código de entrada sobre el que se realizarán los análisis.
- Análisis Léxico: Reconoce los tokens que forman parte del código fuente y genera una tala de símbolos que muestra los que reconoció, y también puede identificar tokens inválidos e identificar el tipo de error léxico.
- Análisis Sintáctico: Toma el Token generado por el análisis léxico y genera un árbol de sintaxis utilizando ANTLR4 y lo recorre buscando posibles errores, como podrían ser errores de parsing.
- Análisis semántico: Recorre el AST para corroborar que las variables o funciones definidas en el código fuente se estén utilizando de la manera correcta, por ejemplo, no guardar palabras en una variable entera, y devuelve una tabla de símbolos indicando donde se encuentra y otros datos, como si son de ámbito global, o si pertenecen a otra variable o función.
- Generación de Código Intermedio: Genera un Código de tres direcciones en base al AST, para transformar lo que está escrito en lenguaje de C++ en operaciones más elementales.
- Optimización: Implementa técnicas de optimización para disminuir el tamaño en pantalla del código de tres direcciones, las técnicas implementadas son:
  - Propagación de Constantes
  - Eliminación de Código muerto
  - Simplificación de Expresiones
- Salida Final: Como último paso genera dos archivos de texto, uno para mostrar la generación de Código intermedio y otro para mostrar la optimización de dicho código.

# Implementación

El compilador fue desarrollado en Java con ANTLR4 para la generación del analizador léxico y sintáctico, a continuación, son las principales clases que forman parte del proyecto:

- App.java: Punto de entrada que coordina todas las fases de compilación.
- AnalizadorLexico.java: Gestión del análisis léxico y detección de errores.
- AnalizadorSintactico.java: Construcción del AST y verificación sintáctica.
- AnalizadorSemantico.java: Verificación de tipos, ámbitos y consistencia semántica.
- GeneradorCodigoIntermedio.java: Generación de código de tres direcciones.
- Optimizador.java: Aplicación de técnicas de optimización.
- TablaSimbolos.java: Gestión de ámbitos y símbolos.

La gramática definida en el archivo compilador.g4 implementa un subconjunto de C++ con las siguientes características:

#### **Estructura del programa:**

programa: (declaracion\_global | definicion\_funcion)\* definicion\_funcion\_main  
EOF;

definicion\_funcion\_main: INT MAIN PA PC bloque;

definicion\_funcion: tipo IDENTIFICADOR PA parametros? PC bloque;

#### **Tipos de datos soportados:**

tipo: INT | CHAR | DOUBLE | VOID | BOOL;

#### **Estructuras de Control:**

IF PA expresion PC instruccion (ELSE instruccion)?;

WHILE PA expresion PC instruccion;

FOR PA (declaracion\_variable | asignacion\_simple)? PyC expresion? PyC  
(asignacion\_simple | expresion)? PC instruccion;

#### **Sistema de Expresiones:**

expresion:

expresion OR expresion

| expresion AND expresion

| expresion (EQ | NEQ | LT | LE | GT | GE) expresion

| expresion (SUMA | RESTA) expresion

| expresion (MULT | DIV | MOD) expresion

| (SUMA | RESTA | NOT) expresion

| IDENTIFICADOR PA argumentos? PC

| referencia

| literales

| PA expresion PC;

La clase TablaSimbolos.java implementa una estructura jerárquica con pila de ámbitos:

#### Características Principales:

- Ámbitos anidados: Soporte para bloques anidados mediante pila.
- Búsqueda jerárquica: Las variables se buscan desde el ámbito actual hacia el global.
- Registro completo: Todas las variables se mantienen para análisis de uso.
- Funciones separadas: Mapa especializado para funciones.

#### Algoritmos utilizados en cada fase del compilador

##### Analizador Léxico:

- Uso de compiladorLexer generador por ANTLR4 para la tokenización.
- Detección de errores utilizando patrones regex para identificadores inválidos, decimales mal formados.

##### Analizador Sintáctico:

- Construcción del AST utilizando ParseTree generado por ANTLR4
- Manejo de errores utilizando un manejador de errores personalizado, y separando un default para recuperación y un bail para fallas rápidas.

Analizador Semántico, Implementamos un sistema que realiza dos pasadas:

- En la primera pasada registra las funciones.  
“private void registrarFunciones(ParseTree arbol)”
- En la segunda hace una verificación de que la semántica sea correcta haciendo tareas como, verificar los tipos de las variables, controlar en que ámbito se encuentra, etc.  
“private String  
analizarTipoExpresion(compiladorParser.ExpresionContext ctx)”

Generación de Código Intermedio:

- Una traducción dirigida por sintaxis donde cada nodo del AST genera código específico.
- Un Algoritmo de numeración secuencial, (t0, t1, t2, ...)
- Un Algoritmo para saltos y control de flujo.
- Un Algoritmo con stacks para manejar breaks y continuos anidados.

Optimización:

- Algoritmos para cumplir con los 3 tipos de optimización implementados.
- Propagación de constantes: “private List<String>  
propagacionConstantes()”
- Eliminación de Código Muerto: “private Set<String>  
identificarVariablesNecesarias(List<String> codigo)”
- Simplificación de Expresiones: “private String  
simplificarIdentidad(String destino, String op1, String operador,  
String op2)”

## Ejemplos y Pruebas

```
1 // Archivo de prueba SIN errores - SUPER SIMPLE
2 // Compatible con la gramática básica
3
4 // Variables globales
5 int contadorGlobal;
6 double valorPi;
7 char inicial;
8 bool activo;
9
10 // Función simple que retorna valor
11 int sumar(int a, int b) {
12     int resultado;
13     resultado = a + b;
14     contadorGlobal = contadorGlobal + 1;
15     return resultado;
16 }
17
18 // Función main
19 int main() {
20     int estado;
21     int temp;
22     int numeros[3];
23
24     // Inicializar variables globales
25     contadorGlobal = 0;
26     valorPi = 3.14;
27     inicial = 'M';
28
29     // Usar arrays
30     numeros[0] = 10;
31     numeros[1] = 20;
32     numeros[2] = 30;
33
34     // Operaciones básicas
35     temp = numeros[0] + numeros[1];
36     temp = temp * 2;
37     temp = temp / 3;
38     temp = temp % 5;
39
40     // Usar función que retorna valor
41     estado = sumar(temp, 5);
42
43     // Usar variables globales
44     contadorGlobal = estado;
45     valorPi = temp;
46     inicial = 'X';
47
48     // Estructuras de control básicas
49     if (estado > 0) {
50         int auxiliar;
51         auxiliar = estado + 10;
52         estado = auxiliar;
53     }
54
55     return estado;
56 }
```

Utilizamos este código de entrada para nuestras pruebas con el compilador.

¿ TABLA DE TOKENS:

| Nº | LEXEMA         | TIPO          | LÍN | COL | ESTADO |
|----|----------------|---------------|-----|-----|--------|
| 1  | int            | INT           | 5   | 1   | ¿ OK   |
| 2  | contadorGlobal | IDENTIFICADOR | 5   | 5   | ¿ OK   |
| 3  | ;              | PyC           | 5   | 19  | ¿ OK   |
| 4  | double         | DOUBLE        | 6   | 1   | ¿ OK   |
| 5  | valorPi        | IDENTIFICADOR | 6   | 8   | ¿ OK   |
| 6  | ;              | PyC           | 6   | 15  | ¿ OK   |
| 7  | char           | CHAR          | 7   | 1   | ¿ OK   |
| 8  | inicial        | IDENTIFICADOR | 7   | 6   | ¿ OK   |
| 9  | ;              | PyC           | 7   | 13  | ¿ OK   |
| 10 | bool           | BOOL          | 8   | 1   | ¿ OK   |
| 11 | activo         | IDENTIFICADOR | 8   | 6   | ¿ OK   |
| 12 | ;              | PyC           | 8   | 12  | ¿ OK   |
| 13 | int            | INT           | 11  | 1   | ¿ OK   |
| 14 | sumar          | IDENTIFICADOR | 11  | 5   | ¿ OK   |
| 15 | (              | PA            | 11  | 10  | ¿ OK   |
| 16 | int            | INT           | 11  | 11  | ¿ OK   |
| 17 | a              | IDENTIFICADOR | 11  | 15  | ¿ OK   |
| 18 | ,              | COM           | 11  | 16  | ¿ OK   |
| 19 | int            | INT           | 11  | 18  | ¿ OK   |
| 20 | b              | IDENTIFICADOR | 11  | 22  | ¿ OK   |
| 21 | )              | PC            | 11  | 23  | ¿ OK   |
| 22 | {              | LA            | 11  | 25  | ¿ OK   |
| 23 | int            | INT           | 12  | 5   | ¿ OK   |
| 24 | resultado      | IDENTIFICADOR | 12  | 9   | ¿ OK   |
| 25 | ;              | PyC           | 12  | 18  | ¿ OK   |
| 26 | resultado      | IDENTIFICADOR | 13  | 5   | ¿ OK   |
| 27 | =              | IGU           | 13  | 15  | ¿ OK   |
| 28 | a              | IDENTIFICADOR | 13  | 17  | ¿ OK   |
| 29 | +              | SUMA          | 13  | 19  | ¿ OK   |
| 30 | b              | IDENTIFICADOR | 13  | 21  | ¿ OK   |
| 31 | ;              | PyC           | 13  | 22  | ¿ OK   |
| 32 | contadorGlobal | IDENTIFICADOR | 14  | 5   | ¿ OK   |
| 33 | =              | IGU           | 14  | 20  | ¿ OK   |
| 34 | contadorGlobal | IDENTIFICADOR | 14  | 22  | ¿ OK   |
| 35 | +              | SUMA          | 14  | 37  | ¿ OK   |
| 36 | 1              | ENTERO        | 14  | 39  | ¿ OK   |
| 37 | ;              | PyC           | 14  | 40  | ¿ OK   |
| 38 | return         | RETURN        | 15  | 5   | ¿ OK   |
| 39 | resultado      | IDENTIFICADOR | 15  | 12  | ¿ OK   |
| 40 | ;              | PyC           | 15  | 21  | ¿ OK   |
| 41 | }              | LC            | 16  | 1   | ¿ OK   |
| 42 | int            | INT           | 19  | 1   | ¿ OK   |
| 43 | main           | MAIN          | 19  | 5   | ¿ OK   |
| 44 | (              | PA            | 19  | 9   | ¿ OK   |
| 45 | )              | PC            | 19  | 10  | ¿ OK   |
| 46 | {              | LA            | 19  | 12  | ¿ OK   |
| 47 | int            | INT           | 20  | 5   | ¿ OK   |
| 48 | estado         | IDENTIFICADOR | 20  | 9   | ¿ OK   |
| 49 | ;              | PyC           | 20  | 15  | ¿ OK   |
| 50 | int            | INT           | 21  | 5   | ¿ OK   |
| 51 | temp           | IDENTIFICADOR | 21  | 9   | ¿ OK   |
| 52 | ;              | PyC           | 21  | 13  | ¿ OK   |
| 53 | int            | INT           | 22  | 5   | ¿ OK   |

El analizador léxico genera una tabla de tokens de los elementos que forman parte del código de entrada

¿ ESTADÍSTICAS DETALLADAS:

```

¿ Total de tokens analizados: 167
¿ Tokens válidos: 167
¿ Tokens con error: 0

¿ ¡ANÁLISIS LÉXICO COMPLETADO EXITOSAMENTE!
  ¿ No se encontraron errores léxicos
  ¿ Todos los tokens fueron reconocidos correctamente
  ¿ El código puede proceder al análisis sintáctico
¿ Análisis léxico completado sin errores. Procediendo...

```

Y una lista de estadísticas del resultado del análisis.

## === 2. ANÁLISIS SINTÁCTICO ===

### ANÁLISIS SINTÁCTICO C++ Técnicas de Compilación

? Archivo analizado: input/Ejemplo-Final-Correcto.txt

? RESUMEN DEL ANÁLISIS:

? ¡ANÁLISIS SINTÁCTICO EXITOSO!

? El programa tiene una estructura sintáctica válida

? ANÁLISIS DETALLADO DEL PROGRAMA:

? CONSTRUCCIONES DETECTADAS:

? Expresiones : 44

? Condicionales if : 1

? Profundidad máxima del AST: 14

? Análisis sintáctico completado sin errores. Procediendo...

Los resultados del analizados sintáctico indicando si fue exitoso o no y mostrando cuantas construcciones detecto.

#### === 4. ANÁLISIS SEMÁNTICO ===

##### ANÁLISIS SEMÁNTICO C++ Técnicas de Compilación

? Archivo analizado: input/Ejemplo-Final-Correcto.txt

? RESUMEN GENERAL:

? ¡ANÁLISIS SEMÁNTICO EXITOSO!

? El programa es semánticamente correcto

? Errores críticos: 0

? Warnings: 1

? Tiempo de análisis: 11 ms

?? WARNINGS (RECOMENDACIONES):

? Conversión implícita de tipos (1 ocurrencias):

? Línea 45, Col 15: Conversión implícita de int a double

? Use conversión explícita para evitar pérdida de precisión

#### === TABLA DE SÍMBOLOS ===

| NOMBRE         | TIPO   | CATEGORÍA | LÍNEA | COLUMNA | ÁMBITO | DETALLES             |
|----------------|--------|-----------|-------|---------|--------|----------------------|
| contadorGlobal | int    | variable  | 5     | 5       | global | [private]            |
| valorPi        | double | variable  | 6     | 8       | global | [private]            |
| inicial        | char   | variable  | 7     | 6       | global | [private]            |
| activo         | bool   | variable  | 8     | 6       | global | [private]            |
| sumar          | int    | función   | 11    | 1       | global | [private] [int, int] |
| a              | int    | parametro | 11    | 11      | sumar  | [private]            |
| b              | int    | parametro | 11    | 18      | sumar  | [private]            |
| resultado      | int    | variable  | 12    | 9       | sumar  | [private]            |
| main           | int    | funcion   | 19    | 1       | global | [private]            |
| estado         | int    | variable  | 20    | 9       | main   | [private]            |
| temp           | int    | variable  | 21    | 9       | main   | [private]            |
| numeros        | int    | variable  | 22    | 9       | main   | [arr:3] [private]    |
| auxiliar       | int    | variable  | 50    | 13      | main   | [private]            |

? SUGERENCIAS DE MEJORA:

?? Elimine 1 variable(s) que no se utilizan para limpiar el código

? Inicialice 1 variable(s) al declararlas para evitar errores

?? Revise los 1 warnings para mejorar la calidad del código

? Análisis semántico completado sin errores.

?? Se encontraron 1 warnings (no críticos).

Los resultados del análisis semántico, indicando errores críticos y advertencias.

Recomendaciones para solucionar las advertencias.

Y una tabla de símbolos indicando cosas como cual es su categoría, en qué línea se encuentra, el ámbito de la variable etc.

Y unas sugerencias de mejora de las advertencias.

```

=== 5. GENERACIÓN DE CÓDIGO INTERMEDIO ===
? Iniciando recorrido del AST con GeneradorCodigoIntermedio...

? Código de tres direcciones generado:

0: // Código de tres direcciones generado
1: PROGRAMA_INICIO:
2: // Declaración de variables globales
3: DECLARE contadorGlobal int
4: DECLARE valorPi double
5: DECLARE inicial char
6: DECLARE activo bool
7: func_sumar:
8: PARAM a int
9: PARAM b int
10: DECLARE resultado int
11: t0 = a + b
12: resultado = t0
13: t1 = contadorGlobal + 1
14: contadorGlobal = t1
15: return resultado
16: func_main:
17: DECLARE estado int
18: DECLARE temp int
19: DECLARE numeros[3] int
20: contadorGlobal = 0
21: valorPi = 3.14
22: inicial = 'M'
23: numeros[0] = 10
24: numeros[1] = 20
25: numeros[2] = 30
26: t2 = numeros[0] + numeros[1]
27: temp = t2
28: t3 = temp * 2
29: temp = t3
30: t4 = temp / 3
31: temp = t4
32: t5 = temp % 5
33: temp = t5
34: CALL func_sumar, temp, 5
35: t6 = RETURN_VALUE
36: estado = t6
37: contadorGlobal = estado
38: valorPi = temp
39: inicial = 'X'
40: t7 = estado > 0
41: tt8 = t7
42: if t8 goto THEN_0
43: goto END_IF_0
44: THEN_0:
45: DECLARE auxiliar int
46: t9 = estado + 10
47: auxiliar = t9
48: estado = auxiliar
49: END_IF_0:
50: return estado
51: PROGRAMA_FIN:

```

La generación de código intermedio muestra el código de tres direcciones generado.

y donde se guardó el código generado

? Código intermedio guardado en: ejemplo-final-correcto\_codigo\_intermedio.txt

=== 6. OPTIMIZACIÓN DE CÓDIGO ===

? Aplicando optimizaciones al código intermedio...

#### REPORTE DE OPTIMIZACIÓN

? INICIANDO OPTIMIZACIÓN DE CÓDIGO

Código original: 52 instrucciones

? PROPAGACIÓN DE CONSTANTES:

? Optimizaciones aplicadas: 0

? SIMPLIFICACIÓN DE EXPRESIONES:

? Simplificaciones aplicadas: 0

? ELIMINACIÓN DE ASIGNACIONES REDUNDANTES:

? t0 = a + b + resultado = t0 ? resultado = a + b

? t1 = contadorGlobal + 1 + contadorGlobal = t1 ? contadorGlobal = contadorGlobal + 1

? t2 = numeros[0] + numeros[1] + temp = t2 ? temp = numeros[0] + numeros[1]

? t3 = temp \* 2 + temp = t3 ? temp = temp \* 2

? t4 = temp / 3 + temp = t4 ? temp = temp / 3

? t5 = temp % 5 + temp = t5 ? temp = temp % 5

? t6 = RETURN\_VALUE + estado = t6 ? estado = RETURN\_VALUE

? t7 = estado > 0 + tt8 = t7 ? tt8 = estado > 0

? t9 = estado + 10 + auxiliar = t9 ? auxiliar = estado + 10

? Optimizaciones aplicadas: 9

?? ELIMINACIÓN DE CÓDIGO MUERTO:

? Asignaciones eliminadas: 0

? OPTIMIZACIÓN COMPLETADA

Optimizaciones aplicadas: 9

Reducción: 17,3%

? RESUMEN FINAL:

Instrucciones originales: 52

Instrucciones optimizadas: 43

Total optimizaciones: 9

Reducción: 17,3%

Un reporte de las cosas que fueron optimizados en base al código de tres direcciones original, indicando cuantas optimizaciones se realizaron.

? Código optimizado:

? CÓDIGO OPTIMIZADO:

```
1 // Código de tres direcciones generado
2 PROGRAMA_INICIO:
3 // Declaración de variables globales
4 DECLARE contadorGlobal int
5 DECLARE valorPi double
6 DECLARE inicial char
7 DECLARE activo bool
8 func_sumar:
9 PARAM a int
10 PARAM b int
11 DECLARE resultado int
12 resultado = a + b
13 contadorGlobal = contadorGlobal + 1
14 return resultado
15 func_main:
16 DECLARE estado int
17 DECLARE temp int
18 DECLARE numeros[3] int
19 contadorGlobal = 0
20 valorPi = 3.14
21 inicial = 'M'
22 numeros[0] = 10
23 numeros[1] = 20
24 numeros[2] = 30
25 temp = numeros[0] + numeros[1]
26 temp = temp * 2
27 temp = temp / 3
28 temp = temp % 5
29 CALL func_sumar, temp, 5
30 estado = RETURN_VALUE
31 contadorGlobal = estado
32 valorPi = temp
33 inicial = 'X'
34 tt8 = estado > 0
35 if t8 goto THEN_0
36 goto END_IF_0
37 THEN_0:
38 DECLARE auxiliar int
39 auxiliar = estado + 10
40 estado = auxiliar
41 END_IF_0:
42 return estado
43 PROGRAMA_FIN:
```

? Código optimizado guardado en: ejemplo-final-correcto\_codigo\_optimizado.txt

Muestra el código optimizado y donde se guardó el código optimizado.

```
RESUMEN FINAL

? RESULTADOS POR FASE:

? Análisis Léxico:    ? EXITOSO
  └ Tokens: 167 | Errores: 0 | Éxito: 100,0%
? Análisis Sintáctico: ? EXITOSO
  └ Nodos: 352 | Errores: 0 | Tiempo: 69 ms
? Análisis Semántico:  ? EXITOSO
  └ Errores: 0 | Warnings: 1 | Tiempo: 11 ms

? ;COMPILACIÓN EXITOSA!
? El programa pasó todas las fases de análisis
?? Se encontraron 1 warnings (recomendaciones de mejora)

? TABLA DE SÍMBOLOS:
  Variables: 11 | Funciones: 2 | Ámbitos: 1
```

Y un resumen final con datos relacionado con los resultados de la compilación.

```

1 // Variables globales
2 int variableGlobal;
3 int variableGlobal; // ERROR: Variable duplicada en el mismo ámbito
4 double valorGlobal;
5 char caracterGlobal;
6
7 // Función con múltiples errores
8 int miFuncion(int parametro1, double parametro2) {
9     int variableLocal;
10    int variableLocal; // ERROR: Variable duplicada en función
11
12    // Variables nunca utilizadas (WARNING)
13    int variableNoUsada1;
14    string variableNoUsada2;
15    double variableNoUsada3;
16
17    parametro1 = 100; // OK: asignación a parámetro
18    variableLocal = parametro1 + 5; // OK: uso de variable
19
20    // ERROR: Asignación a variable no declarada
21    variableFantasma = 42;
22
23    // ERROR: Asignación a una función (no es variable)
24    miFuncion = 10;
25
26    // OK: usar variable global
27    valorGlobal = 3.14;
28
29    return variableLocal;
30 }
31
32 void funcionVoid() {
33     int x;
34     int y;
35     int z; // WARNING: declarada pero no usada
36
37     x = 10; // OK
38     y = x + 5; // OK: uso de variables
39
40     // ERROR: variable no declarada
41     w = x + y;
42 }
43
44 // Función main para completar el programa
45 int main() {
46     int resultado;
47     int valor;
48
49     resultado = miFuncion(5, 3.14); // OK: uso de función
50     valor = resultado + 10; // OK: uso de variables
51
52     // ERROR: variable no declarada
53     variableFinal = valor;
54
55     // OK: usar variable global
56     variableGlobal = valor;
57
58     return resultado;
59 }

```

Código de entrada con errores para demostrar como funciona cuando se encuentra con un error.

#### === 4. ANÁLISIS SEMÁNTICO ===

? Tabla de símbolos construida:

##### === TABLA DE SÍMBOLOS ===

| NOMBRE           | TIPO   | CATEGORÍA | LÍNEA | COLUMNA | ÁMBITO      | DETALLES                |
|------------------|--------|-----------|-------|---------|-------------|-------------------------|
| variableGlobal   | int    | variable  | 2     | 5       | global      | [private]               |
| valorGlobal      | double | variable  | 4     | 8       | global      | [private]               |
| caracterGlobal   | char   | variable  | 5     | 6       | global      | [private]               |
| miFuncion        | int    | funcion   | 8     | 1       | global      | [private] [int, double] |
| parametro1       | int    | parametro | 8     | 15      | miFuncion   | [private]               |
| parametro2       | double | parametro | 8     | 31      | miFuncion   | [private]               |
| variableLocal    | int    | variable  | 9     | 9       | miFuncion   | [private]               |
| variableNoUsada1 | int    | variable  | 13    | 9       | miFuncion   | [private]               |
| variableNoUsada2 | string | variable  | 14    | 12      | miFuncion   | [private]               |
| variableNoUsada3 | double | variable  | 15    | 12      | miFuncion   | [private]               |
| funcionVoid      | void   | funcion   | 32    | 1       | global      | [private] []            |
| x                | int    | variable  | 33    | 9       | funcionVoid | [private]               |
| y                | int    | variable  | 34    | 9       | funcionVoid | [private]               |
| z                | int    | variable  | 35    | 9       | funcionVoid | [private]               |
| main             | int    | funcion   | 45    | 1       | global      | [private] []            |
| resultado        | int    | variable  | 46    | 9       | main        | [private]               |
| valor            | int    | variable  | 47    | 9       | main        | [private]               |

##### ? ERRORES SEMÁNTICOS:

- ? Error: La variable global 'variableGlobal' ya está declarada (línea 3, columna 5)
- ? Error: La variable 'variableLocal' ya está declarada en este ámbito (línea 10, columna 9)
- ? Error: La variable 'variableFantasma' no está declarada (línea 21, columna 5)
- ? Error: No se puede asignar valor a 'miFuncion' porque no es una variable (línea 24, columna 5)
- ? Error: La variable 'w' no está declarada (línea 41, columna 5)
- ? Error: La variable 'variableFinal' no está declarada (línea 53, columna 5)

##### ?? WARNINGS SEMÁNTICOS:

- ?? Warning: La variable 'variableNoUsada3' se declara pero no se utiliza (línea 15, columna 12)
- ?? Warning: La variable 'variableNoUsada1' se declara pero no se utiliza (línea 13, columna 9)
- ?? Warning: La variable 'variableNoUsada2' se declara pero no se utiliza (línea 14, columna 12)
- ?? Warning: La variable 'z' se declara pero no se utiliza (línea 35, columna 9)
- ?? El código tiene warnings, pero se puede continuar.

?? Se encontraron 6 errores semánticos.

##### ?? GENERACIÓN DE CÓDIGO INTERMEDIO/OPTIMIZACIÓN OMITIDA

Motivo: Errores en fases anteriores impiden la generación

El cambio principal se encuentra en el análisis semántico, en el cual se muestran los errores, los cuales se muestran en tojo, y unas advertencias, y en los casos de un error semántico, se cancela la generación de código intermedio.

## RESUMEN FINAL

### ? RESULTADOS POR FASE:

#### ? Análisis Léxico: ? EXITOSO

└ Tokens: 132 | Errores: 0 | Éxito: 100,0%

#### ? Análisis Sintáctico: ? EXITOSO

└ Nodos: 285 | Errores: 0 | Tiempo: 63 ms

#### ? Análisis Semántico: ? CON ERRORES

└ Errores: 6 | Warnings: 4 | Tiempo: 13 ms

### ? COMPILACIÓN FALLIDA

#### ? Errores semánticos detectados

? Compilación detenida debido a errores semánticos.

### ? TABLA DE SÍMBOLOS:

Variables: 14 | Funciones: 3 | Ámbitos: 1

Y una lista de resumen final en el que nuevamente se muestran el número de errores.

## Conclusión

Pudimos implementar un compilador completo capaz de realizar los 3 análisis, léxico, sintáctico y semántico, tomar ese análisis generar un código de tres direcciones y optimizar dicho código.

Link al repositorio en GitHub:

<https://github.com/Juanimelillo159/tptc.git>

## Manual de Usuario:

En este manual explicare paso por paso lo que un usuario que no conoce nada de un compilador debe hacer para poder compilar el código que desee.

Considerando que tenga Visual Estudio Code instalado con la extensión de ANTLR4.

1. Ingresar su código fuente en la carpeta titulada input, preferiblemente como un archivo de texto.
2. Una vez ingresado el código, ir al archivo titulado App.java.
3. En la línea número 13 del archivo App.java, cambiar el nombre del archivo a analizar por el nombre del archivo que se desee compilar.
4. Hacer correr el archivo App.java, y los resultados se mostrarán por consola.